This, super and Final

Module-2



Dr.M. Sivakumar

Professor & Technical Trainer / CSE

Saveetha School of Engineering, SIMATS

sivakumarm.sse@saveetha.com

+91-9500600868

‘this’ keyword

- **‘this’** is a reference variable that refers to the current object.
- Can access any instance variable and method by using **this** keyword.
- The main purpose is to solve the confusion when we have same variable name for instance and local variables.

Syntax

```
this.data member of current class.
```

- By using ‘this’ keyword, we can call the attributes, methods and constructors of the current class.

‘this’ keyword

Calling instance variable

Example of this keyword in java

```
class Employee
{
   int id;
   String name;

   Employee(int id,String name)
  {
   this.id = id;
   this.name = name;
  }
```

```
      void show()
  {
   System.out.println(id+" "+name);
  }
   public static void main(String args[])
   {
    Employee e1 = new Employee(111,"Harry");
    Employee e2 = new Employee(112,"Jacy");
    e1.show();
    e2.show();
   }
}
```

Output

```
111 Harry
112 Jacy
```

‘this’ keyword

Calling a constructor

```
class Demo
{
      Demo ()
      {
            // Calling constructor
           this(“I am a constructor”);
      }
      Demo(String str){
            System.out.println(str);
      }
      public static void main(String[] args) {
              Demo d = new Demo();
      }
}
```

Calling a method

```
class Demo
{
        public void getName()
        {
         System.out.println(“I am a method”);
        }
        public void display()
        {
         this.getName();  // Calling a method
        }
        public static void main(String[] args) {
              Demo d = new Demo();
              d.display();
        }
}
```

‘this’ keyword

Note:

- The scope of "**this**" keyword is within the class.
- The main purpose of using "**this**" keyword in real life application is to differentiate variable of class or formal parameters of methods or constructor

‘super’ keyword

- ‘super’ Keyword is a **reference** variable that is used in inheritance to refer to the immediate parent class object from the child class.

Syntax

```
super.baseclass features.
```

Uses

- To access the parent class variable
- To invoke the parent class method
- To invoke the parent class constructors with and without argument.

‘super’ keyword

Calling Variable

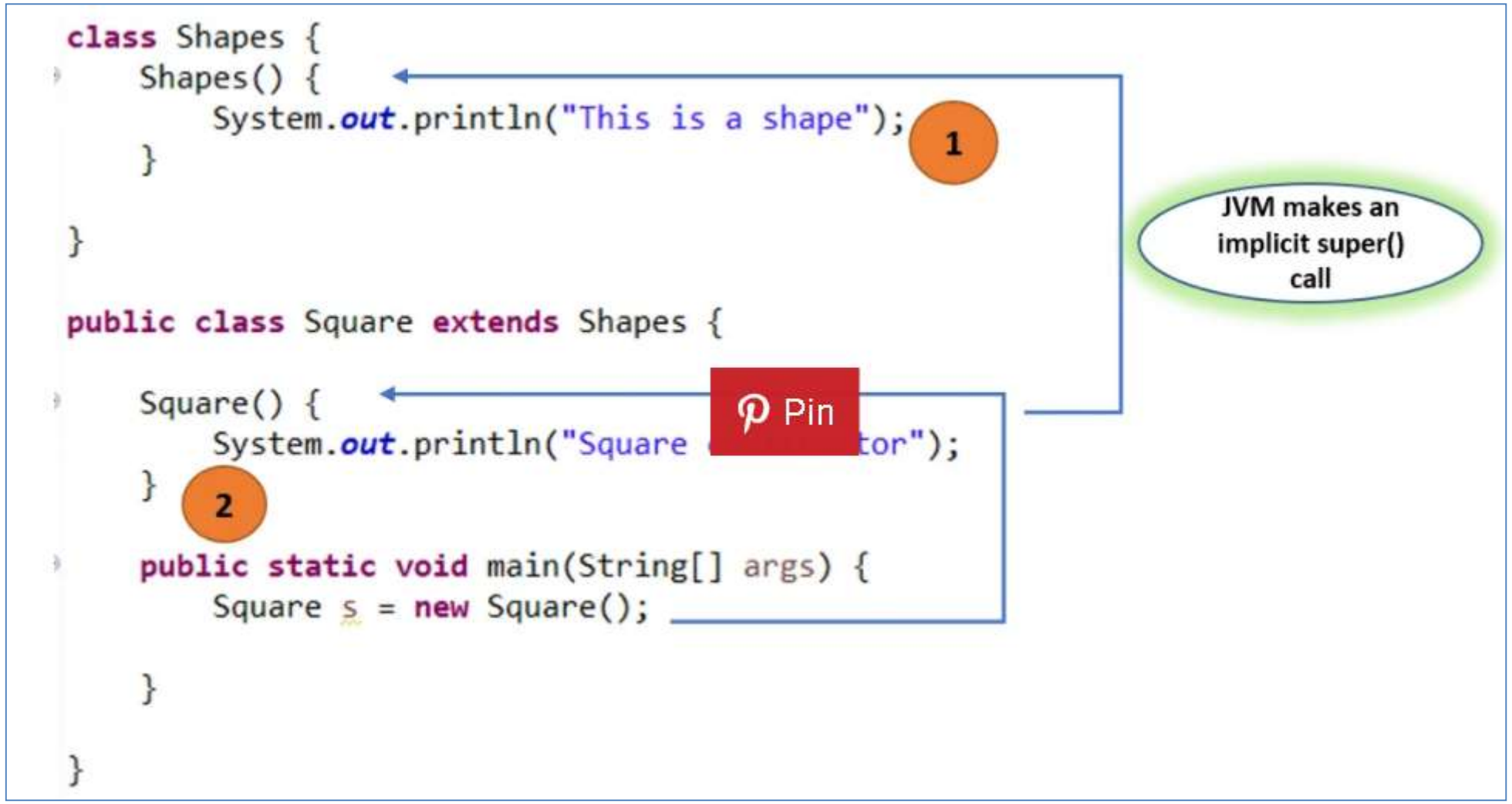
```
class Employee {
String role = "Software Engineer";
}
public class Developer extends Employee {
String role = "Developer";
public void displayRole() {
System.out.println(role);
System.out.println(super.role);
}
public static void main(String[] args) {
Developer d = new Developer();
d.displayRole(); }
}
```

Calling method

```
class Employee {
String role = "Software Engineer";
public void displayRole() {
System.out.println("Employee Role: " + role);
}
}
public class Developer extends Employee {
String role = "Developer";
public void displayRole() {
System.out.println(role);
super.displayRole();
}
public static void main(String[] args) {
Developer d = new Developer();
d.displayRole();
}
}
```

‘super’ keyword

Calling constructor



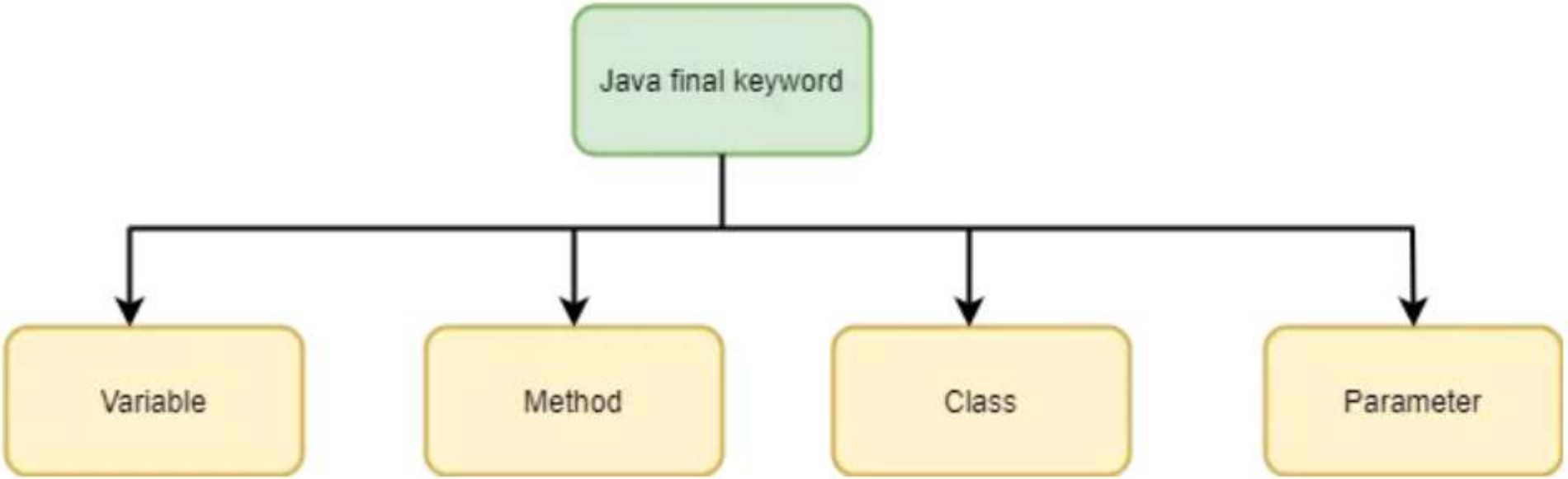
‘super’ keyword

Calling Parameterized constructor

```
class Shapes {
        Shapes() {
        System.out.println("This is a shape");
        }
        Shapes(String name) {
        System.out.println("Shape name is: " + name);
        }
        }
        public class Square extends Shapes {
        Square() {
        super("Square");    // calls base class parameterized constructor
        System.out.println("Square constructor");
        }
        public static void main(String[] args) {
        Square s = new Square();
        }
}
```

‘final’ keyword

- It is used to make a variable as a constant, restrict method overriding and restrict inheritance.
- In java language final keyword can be used in following way



‘final’ variable

- Final keyword is used to make a variable as a constant.
- A variable declared with the final keyword cannot be modified by the program after initialization



Final Keyword in java Example

```
public class Circle
{
public  static final double PI=3.14159;

public static void main(String[] args)
{
System.out.println(PI);
}
}
```

‘final’ method

- It makes a method final, meaning that sub classes can not override this method.
- The compiler checks and gives an error if you try to override the method.



Example

```
class Employee
{
final void disp()
{
System.out.println("Hello Good Morning");
}
}
class Developer extends Employee
{
void disp()
{
System.out.println("How are you ?");
}
}
```

```
class FinalDemo
{
public static void main(String args[])
{
Developer obj=new Developer();
obj.disp();
}
}
```

‘final’ class

- It makes a class final, meaning that the class can not be inheriting by other classes.
- When we want to restrict inheritance then make class as a final.



Example

```
public final class A
{
.....
.....
}
public class B extends  A
{
// it gives an error, because we can not inherit final class
}
```